

Caching

1 Cache structure

1. A 64 KiB direct-mapped cache has 16-byte blocks. You may assume memory is byte-addressed (so 1 word = 1 byte). If addresses are 32 bits long, how many bits are used for the tag, the index, and the offset in this cache?
2. Adapt your answer if the cache is 4-way set associative instead of direct-mapped.
3. How many bits do we need for the index if the cache is fully associative? Explain your answer.

2 Hit/miss in a 2-way LRU cache

An 8-byte, 2-way set-associative cache (with LRU replacement) with 2-byte blocks receives requests for the following (binary) addresses:

0110, 0000, 0010, 0001, 0011, 0100, 1001, 0000, 1010, 1111, 0111.

After each access, give the contents of the cache by filling in the table below; for each set you must show which tags are present.

Address	Set 0	Set 1	Hit/Miss
0110	empty/empty	01/empty	Hit/Miss
0000			Hit/Miss
0010			Hit/Miss
0001			Hit/Miss
0011			Hit/Miss
0100			Hit/Miss
1001			Hit/Miss
0000			Hit/Miss
1010			Hit/Miss
1111			Hit/Miss
0111			Hit/Miss

3 Cache hit rate

3M decides to make Post-its by printing yellow squares on white paper, where for each point in the square the CMYK (cyan, magenta, yellow, black) value must be filled in. 3M hires you to examine three implementations on a machine with a 2048-byte direct-mapped data cache with 32-byte blocks/lines. You are given the following definitions:

```
struct point_color {
    int c;
    int m;
    int y;
    int k;
};

struct point_color square[16][16];

register int i, j;
```

Assume furthermore:

- `sizeof(int) = 4`,
- `square` starts at memory address 0,
- the cache is initially empty,
- the only memory accesses are to entries of the array `square`. The variables `i` and `j` are stored in registers,
- memory is 4-byte addressed, so 1 int = 1 word.

What percentage of the writes in the following code cause a cache **hit**?

Method 1:

```
for (i=0; i<16; i++) {
    for (j=0; j<16; j++) {
        square[i][j].c = 0;
        square[i][j].m = 0;
        square[i][j].y = 1;
        square[i][j].k = 0;
    }
}
```

Method 2:

```
for (i=0; i<16; i++) {
    for (j=0; j<16; j++) {
        square[j][i].c = 0;
        square[j][i].m = 0;
        square[j][i].y = 1;
        square[j][i].k = 0;
    }
}
```

Method 3:

```
for (i=0; i<16; i++) {
    for (j=0; j<16; j++) {
        square[j][i].y = 1;
    }
}

for (i=0; i<16; i++){
    for (j=0; j<16; j++) {
        square[j][i].c = 0;
        square[j][i].m = 0;
        square[j][i].k = 0;
    }
}
```

4 Cache replacement algorithms

For the tree-based and MRU-based pseudo-LRU algorithms, check the following two things and prove them if needed:

1. Suppose w_i is the i -th most recently used word, for $i = 1$ to the set size k . With true LRU, only word w_k can be replaced. Which words can be replaced if we use pseudo-LRU?
2. Suppose a certain block B (i.e., a certain line) is no longer used from some moment on. How long can block B remain present in the cache? In other words: how many replacements can B maximally “survive”?

5 Access times and hit rates

Let T_1 be the access time if a word is found in the L1 cache. If the word is only present in L2 (not in L1), then an extra time T_2 is needed to move the word to L1. Finally, T_M is needed to move a word from main memory to L2. Assume $T_1 = 20$ ns, $T_2 = 100$ ns and $T_M = 400$ ns.

1. If we have hit rates 0.9 and 0.5 in the L1 and L2 caches respectively, what is the average access time?
2. If the hit rate in L2 is 0.6, what must the hit rate in L1 be if the average access time equals 33 ns?

6 Direct mapping versus set-associative mapping with random replacement

Consider the following code fragment from the theory slides:

```
float a[1024], b[1024];

for (i=0; i<1024; i++)
    sum += a[i] * b[i];
```

Assume:

- Cache size: 4 KByte, Block size: 32 byte, Word size: 4 byte
- float: 4 byte
- b is located immediately after a in memory
- i and sum are registers

1. What is the hit rate if we assume direct mapping?
2. If we assume a 4-way set-associative memory with random replacement, compute the (approximate) hit rate.
3. Adapt your answers to the previous two questions if you know that both arrays contain only 256 elements. Do the same for 1536 elements.

7 Benchmarking

Draw a graph that illustrates a cache benchmark as described on p.31 of the theory course. We consider a system that only has an L1 cache; any TLB and paging effects do not need to be shown. The L1 cache size is 32 KByte, a cache line contains 64 bytes, and 8-way associativity is assumed. A hit has an access time of 50 ns; a miss costs an additional 500 ns. Make the graph with stride size (logarithmic scale) on the x-axis and access time on the y-axis, for array sizes from 4 to 512 KByte.

Memory Management

1 Buddy system

1. Consider a buddy system of size 1 MByte. Show the state of memory graphically after the following actions, assuming memory is initially empty:
 - (a) Request $A = 70$ KByte
 - (b) Request $B = 35$ KByte
 - (c) Request $C = 80$ KByte
 - (d) Return A
 - (e) Request $D = 60$ KByte
 - (f) Return B
 - (g) Return D
2. In the standard buddy system, processes ultimately receive memory blocks of powers of 2. Suppose instead we want to use a Fibonacci sequence.
 - Explain how such a system works using a short example. (Tip: assume the total memory size is a (large) Fibonacci number.)
 - Does this system perform better or worse than the standard buddy system in terms of internal fragmentation? Why (or why not)?

Paging

1 Size of the page tables of a process

Assume we have a 32-bit machine with virtual addresses split into 4 parts of 10, 8, 6 and 8 bits respectively. We use these addresses in a 3-level page table system, where the first 10 bits are used for level 1, the next 8 bits for level 2, etc. A single page table entry is 4 bytes. The system is byte-addressed.

1. What is the page size in this system?
2. Suppose a process needs 256K of memory (starting from address 0). How much space do this process's page tables take?
3. Suppose a process has a code segment of 48K starting from (virtual) address 0x10000000, a 600K data segment starting from address 0x80000000, and a stack segment of 64K starting from address 0xF0000000. How much space does the page table of this process require?

2 Determine the number of levels in a multilevel paging system

Assume a machine has 36-bit virtual addresses and pages of size 8K. A single page entry is 8 bytes long.

1. How many pages does the virtual address space contain?
2. Suppose an average process is 8 GByte in size. How many levels are most suitable if we use a multilevel page table (consider 1 to 3 levels)? Weigh the pros and cons. For the 2-level system, choose the size of a level-2 page table so that it fits exactly in one page. For three levels, use a more or less equal split for each of the fields.

3 Count the number of TLB misses

Consider the following code. Assume the binary code fits on one page and the stack also fits on a single page. You may also assume that when calling the function, the stack page and code page are already in the TLB. Assume an integer is 4 bytes and a page contains 4096 bytes.

```
int a[1024][1024], b[1024][1024], c[1024][1024];

multiply() {
    unsigned i,j,k;
    for(i=0; i< 1024; i++)
        for(j=0; j< 1024; j++)
            for(k=0; k< 1024; k++)
                c[i][j] += a[i][k] * b[k][j];
}
```

Given that the TLB has only 8 entries and uses an LRU replacement policy, how many TLB misses occur during the execution of this code?

4 Demand paging: good or bad

Which of the following programming techniques works well in combination with a demand-paged system?

1. Stack
2. Hash table
3. Sequential search
4. Binary search
5. Pure code
6. Vector operations (e.g. vector multiplication)
7. Indirect addressing

5 Useful interventions for demand paging

Suppose we take some statistics on a demand-paged system. We notice the CPU is used 20%, the paging disk 97.3%, and the other I/O devices only 5%. Which interventions seem effective to improve the system?

1. Install a faster CPU
2. Install a larger paging disk
3. Increase the degree of multiprogramming
4. Decrease the degree of multiprogramming
5. Install more memory
6. Install a faster hard disk
7. Allow prepaging
8. Increase the page size

6 Inverted versus standard page tables

If virtual addresses are 64 bits long, the page size is 1 KByte, the size of physical memory is 64 MByte, and a PTE takes two bytes, what is the total size of the inverted and standard page table (assuming each byte is addressed)? First estimate how many bytes an inverted page table entry needs by determining which fields must be present at minimum and how many bits each needs.

7 Inverted page table

Consider a system with 16KB memory, pages of size 1KB, an inverted page table, and a fully associative TLB with 2 entries and an LRU replacement policy. The hash function is:

$$f(\text{PID}, \text{VPN}) = (10 \cdot \text{PID} + \text{VPN}) \bmod 24.$$

The system needs 0.5 ns to read the TLB, 100 ns to read main memory, and 10 ms to access the hard disk.

Fill in the page request table using the given hash anchor table and the inverted page table (IPT). In the IPT, “T” indicates that there is no next page table entry (PTE). In the TLB

column, give the TLB contents after translation. Use the notation (VPN, Frame number) for each line, and use - if a line is empty. In the columns “Frame number”, “TLB hits”, and “Page fault” you should respectively give the physical frame number, whether there was a TLB hit, and whether it caused a page fault. Finally, in the last column, fill in the time needed to perform the translation. We assume that the buffer can be updated in parallel after translating and executing the next instruction, so you do not need to count time for that. In case of a page fault, you do not need to update the data structures; you only need to give the time required to discover that there is a page fault.

Table 1: Hash anchor table

Index	0	1	2	3	4	5	6	7	8	9	10	11
Value	7	3	12	15	13	6	2	4	0	14	8	5

Index	12	13	14	15	16	17	18	19	20	21	22	23
Value	15	11	10	8	5	7	9	10	12	6	13	1

Table 2: Inverted Page Table (IPT)

Index	0	1	2	3	4	5	6	7
PID	0	3	2	3	0	8	0	2
VPN	8	17	8	4	1	3	5	4
Next PTE	2	T	T	4	T	T	10	12

Index	8	9	10	11	12	13	14	15
PID	2	0	8	6	0	2	2	4
VPN	2	3	13	1	2	32	0	4
Next PTE	3	T	0	9	14	8	T	11

Requests to fill in

PID	VPN	TLB	Frame number	TLB hits	Page fault	Latency (ns)
0	1	Hit/Miss		Yes/No	Yes/No	
0	2	Hit/Miss		Yes/No	Yes/No	
0	1	Hit/Miss		Yes/No	Yes/No	
0	3	Hit/Miss		Yes/No	Yes/No	
8	3	Hit/Miss		Yes/No	Yes/No	
0	3	Hit/Miss		Yes/No	Yes/No	
2	2	Hit/Miss		Yes/No	Yes/No	
2	1	Hit/Miss		Yes/No	Yes/No	
8	13	Hit/Miss		Yes/No	Yes/No	
8	3	Hit/Miss		Yes/No	Yes/No	

8 Count the number of page faults

Consider the following page sequence:

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.

How many page faults occur if this process has 1, 2, $\dots$, 6 or 7 frames available? Do this for the following three replacement algorithms: LRU, FIFO and the MIN algorithm. Initially, there are no pages present in memory.

9 Segmented FIFO

True or false?

1. For all pages in L_1 , they were requested more recently than those in L_2 .
2. Pages in L_1 are in LRU order.
3. Pages in L_2 are in LRU order.
4. The $|L_2| - 1$ most recently used pages are safe (cannot be replaced).
5. The $|L_2|$ most recently used pages are safe.
6. A segmented FIFO system with $|L_1| = |L_2| = 2$ is equivalent to a tree-based pseudo-LRU system with 4 pages.

10 Benchmarking

Draw a graph that illustrates a cache benchmark as described on p.31 of the theory course. We consider a system that has an L1 cache and a TLB cache. The size of the L1 cache is 256 KByte, a cache line contains 32 bytes, and direct mapping is assumed. An L1 cache access takes 250 ns, an L1 cache miss adds 500 ns, and a TLB miss adds another 1000 ns. The TLB is 4-way set associative, contains 128 entries, and the page size is 8192 bytes. Make the graph with stride size (logarithmic scale) on the x-axis and access time on the y-axis, for array sizes from 4 KByte to 4096 KByte.

Uniprocessor scheduling

1 A variant of Highest Response Ratio Next

A variant of HRRN, called *minimax response ratio*, aims to minimize the maximum response ratio. Devise a scheduling policy that achieves this. Use the example of Figure 1; assume we must make a scheduling decision at time t . In which order should tasks t_1 , t_2 and t_3 be executed (starting at time t) so that the maximum response ratio

$$\max_s \left(\frac{w + p_s}{p_s} \right)$$

is as small as possible?

Figure 1: Response ratio as a function of time.

2 Rate Monotonic Scheduling

As mentioned in the course, one can show that RMS always meets deadlines if

$$U = \sum_{j=1}^n \frac{p_j}{T_j} \leq n \left(2^{1/n} - 1 \right).$$

1. Give an example of jobs (at least 2 jobs) that meet their deadlines with RMS and have $U = 1$.
2. Give an example of jobs that **do not** meet their deadlines with RMS and have $U \leq 0.95$.
3. Give an example of jobs that **do not** meet their deadlines with RMS and have $U \leq 0.84$.

Disk Management

1 Random seek

We consider a hard disk with N tracks. Assume requested sectors are randomly distributed over all tracks and independent of each other. Compute the average number of tracks the disk head must move between two requests, by answering:

1. What is the probability that the disk head is at position i and must go to position j ?
2. How many tracks must be moved from i to j ?
3. Combine 1 and 2 by considering all possible combinations of i and j , and work it out.
4. Given that N is large, give an approximation for the number of tracks to move, relative to the total number of tracks.

2 Scheduling algorithms

Assume we have an I/O controller that uses the SSTF algorithm, and the disk head is at the middle track of the disk.

1. Give a sequence of 8 requests such that the disk head must pass the middle of the disk between each request. Give the minimal (in terms of total moved tracks) solution for this problem. (You may write requests relative to the middle track; e.g. if the middle track is 512, you may denote track 511 as -1 .)
2. Compute the total number of track switches for this request pattern.
3. Let the same pattern be handled by a LOOK algorithm, and compute the number of track switches. Compare.

3 RAID: efficient?

Given that you have 12 hard disks of 40GB each, determine for the following RAID configurations how many GB you can store on the RAID system:

- RAID 0+1
- RAID 1+0
- RAID 0+5 (all 3 possible splits)
- RAID 5+0 (all 3 possible splits)

4 RAID: robust?

Given that you have 12 hard disks, determine for the following RAID configurations how many disks may fail at most without the system failing (not necessarily unrecoverably). Then compute the probability that the system remains operational when two hard disks fail.

- RAID 0+1
- RAID 1+0

- RAID 0+5 (all 3 possible splits)
- RAID 5+0 (all 3 possible splits)

Concurrency

1 Producer/consumer problem

Give a possible implementation for the producer/consumer problem with semaphores, assuming the buffer is finite. In other words: if the producer wants to write an item and the buffer is full, it must wait until a spot becomes free again.

2 Concurrency

Consider the following code:

```
const int n = 50;
int tally;

void total() {
    int count;
    for (count = 1; count <= n; count++) {
        tally = tally + 1;
    }
}

void main() {
    tally = 0;
    parbegin(total(), total()); // run in parallel
    cout << tally << endl;
}
```

1. What is the minimum and maximum value that the variable `tally` can have at the end of the program?
2. Suppose there are N parallel instances of `total`. How do these bounds change?

3 Paper review

Reviewing papers is one of the many activities of researchers. The goal is to “check” a paper (including correctness) before it can be published in a journal or presented at a conference. The comment below was submitted in 1965 to the fairly well-regarded journal *Communications of the ACM*. Review this comment (Figure 2) for correctness. *Tip: find an execution order where two processes obtain access to the critical section at the same time.*

Figure 2: Comment submitted to Communications of the ACM.

Comments on a Problem in Concurrent
Programming Control

Dear Editor:

I would like to comment on Mr. Dijkstras solution
[Solution of a problem in concurrent programming control.
Comm ACM 8 (Sept. 1965), 569] to a messy problem that is
hardly academic. We are using it now on a multiple computer
complex.

When there are only two computers, the algorithm may be

simplified to the following:

```
Boolean array b(0, 1); integer k, i, j,  
comment This is the program for computer i, which may be  
either 0 or 1; computer j = i is the other one, 1 or 0;
```

```
C0: b(i) := false;  
C1: if k != i then begin  
C2: if not b(j) then go to C2;  
    else k := i; go to C1  
    end;  
    else critical section;  
  
    b(i) := true;  
    remainder of program;  
    go to C0;  
end
```

Mr. Dijkstra has come up with a clever solution to a really practical problem.

4 Binary semaphores

```
void semWait(semaphore s) {  
    semWaitB(mutex);  
    s--;  
    if (s < 0) {  
        semSignalB(mutex);  
        semWaitB(delay);  
    } else  
        semSignalB(mutex);  
}  
  
void semSignal(semaphore s) {  
    semWaitB(mutex);  
    s++;  
    if (s <= 0)  
        semSignalB(delay);  
    semSignalB(mutex);  
}
```

This code could represent a possible implementation that realizes counting semaphores using binary semaphores. Unfortunately, this code may still cause a problem. Find the problem and modify the code to obtain a correct solution.

5 Mutual exclusion

Given the code below, answer the following questions:

- Is mutual exclusion guaranteed for the critical section? Explain.
- Is it possible to end up in a deadlock? Explain.
- Is starvation possible? Explain.

```

global variables:
    int turn = -1;
    bool busy = false;

thread i:
    while (turn != i) {
        while (busy) {
            turn = i;
        }
        busy = true;
    }

    <critical section>

    busy = false;

```

6 Amusement park model

Consider an attraction in an amusement park consisting of a ride in a 1-person cart through a haunted house. In total the attraction has n carts on a fixed track (the carts cannot overtake each other). To save energy, only full carts drive around; the others wait in the station for passengers. The m visitors walk around for a while, arrive at the haunted house, possibly wait in a FIFO queue for a cart, enter the first available cart, enjoy the ride, get out, and then continue their visit.

If we model both visitors and carts as processes that must synchronize (using semaphores), we must prevent that:

- A cart departs while it is not full.
- A cart departs while a passenger is entering or leaving the cart.
- Passengers exit too early, i.e. they may not leave the cart during the ride.

Given the global variables:

```

cars_avail = semaphore initialized to 0;
passenger_safe[] = array of n semaphores initialized to 0;
ride_done[] = array of n semaphores initialized to 0;

```

Full code for the carts:

```

car_j:
while (turned_on) {
    signal(cars_avail)
    <pick up human i>
    wait(passenger_safe[j])
    <drive around with human i>
    signal(ride_done[j])
    <drop off human i>
    wait(passenger_safe[j])
}

```

Partial code for the passengers:

```

human_i:

```

```
...
<wander around in park>
...
<get into car j>
...
<get scared>
...
<get out of car j>
...
<complete visit to park>
...
```

- Complete the code for the visitors. Only use the known semaphore operations.
- What must be changed if carts can carry k people? Give the adapted code for both carts and visitors. We assume passengers do not communicate and cannot determine how many passengers are already in the cart. (2pt)
- Are there constraints on k and m ? Explain. (1pt)

Cache coherency

1 MESI protocol

A system has two processors, each with a fully associative cache of 4 lines. Both caches use the tree-based replacement algorithm, are initially empty, and the processors use the MESI protocol. Complete the table below. In the requests, R stands for read and W stands for write.

Memory access	Cache 1	Cache 2	Comments
Initial state	-- / I I I I	-- / I I I I	
R1	1-- / E I I I	-- / I I I I	
R2	1-2- / E I E I	-- / I I I I	
W3			
R4			
W5			
R6			
R2			
W2			
W1			
R3			
R1			
R6			
W7			
R1			
W7			
R1			
R6			

Multiprocessor scheduling

1 List Scheduling algorithm

Give a (general) example where LS indeed performs $(2 - 1/m)$ times worse than an optimal algorithm, in terms of the maximum completion time; in other words, show that this upper bound cannot be improved. First try to find an example for $m = 2, 3$, then give a general example.

2 Longest Processing Time First

Give a (general) example where LPT indeed performs $(\frac{4}{3} - \frac{1}{3m})$ times worse than an optimal algorithm, in terms of the maximum completion time; in other words, show that this upper bound cannot be improved. First try to find an example for $m = 2, 3, 4$, then give a general example.